Rapport — Gestion des livraisons et chargement d'un camion

1. Introduction

Ce TP demande de concevoir un programme permettant à une entreprise de livraison de gérer une collection de colis (potentiellement jusqu'à 100 000) et de préparer le chargement d'un camion de capacité limitée en maximisant la valeur transportée, tout en pouvant favoriser les colis prioritaires.

Choix du langage : Python. Le TP autorise C ou Python sans avantage donné à l'un ou l'autre. Python a été retenu car :

- ses structures de données natives (`list`, `dict`, `dataclass`) couvrent exactement les besoins du TP (liste ordonnée, table de hachage) sans code de gestion mémoire manuel, ce qui laisse plus de place à la réflexion algorithmique qu'à la gestion de pointeurs ;
- l'écriture et la relecture du code sont plus rapides, ce qui est important dans un cadre pédagogique où la clarté de la démonstration compte autant que la performance brute ;
- Python reste suffisamment prévisible en complexité (listes = tableaux dynamiques, dict = table de hachage) pour que l'analyse de complexité demandée reste rigoureuse, à condition de bien connaître le fonctionnement interne des structures utilisées — ce que ce rapport détaille pour chaque choix important.

2. Architecture du projet

```
src/
├── models.py        dataclass Colis (structure de données de base)
├── repository.py    ColisRepository : stockage + recherche
├── tri.py            tris par poids / valeur / priorité
├── knapsack.py        sac à dos 0/1 (DP) + variante priorité + glouton
├── camion.py           préparation / affichage du chargement
├── stats.py             statistiques
├── data_loader.py       colis d'exemple + génération aléatoire
├── cli.py                 menu interactif
└── main.py                 point d'entrée
```

`cli.py` orchestre tout : il s'appuie sur `repository.py` pour le stockage, `tri.py` pour l'affichage organisé, `camion.py` (qui appelle `knapsack.py`) pour la préparation du camion, et `stats.py` pour les statistiques.

3. Modélisation des données

Un colis est représenté par une `dataclass Colis` (id, poids, valeur, priorité, destination), validée à la construction (poids entier positif, priorité entre 1 et 5).

Convention de priorité retenue : l'énoncé demande de "favoriser les colis possédant une **priorité importante**". Cette formulation désigne la valeur numérique de la priorité elle-même comme une mesure d'importance : on adopte donc la convention où **plus le nombre est élevé, plus le colis est important** (priorité 5 = la plus importante, priorité 1 = la moins importante). Ce choix est explicité ici car il n'est pas universel : dans un système de tickets, "priorité 1" désigne souvent l'urgence maximale. Le choix inverse serait tout aussi défendable avec une autre formulation de l'énoncé ; ce qui compte est la cohérence du système et sa justification.

Poids et capacité toujours entiers. C'est l'hypothèse qui permet d'utiliser une programmation dynamique exacte pour le sac à dos (section 5.3) — une capacité ou des poids décimaux casseraient l'indexation du tableau `dp` par le poids et nécessiteraient une autre approche (discrétisation, ou renoncer à l'optimalité garantie).

4. Fonctionnalités de base (gestion de la collection)

Fonctionnalité	Implémentation
Ajouter un colis	`ColisRepository.ajouter_colis`
Afficher les colis	`ColisRepository.lister_tous`
Rechercher par ID	`ColisRepository.rechercher_par_id`
Trier par poids / valeur / priorité	`tri.py`

5. Choix algorithmiques majeurs

5.1 Fiche n°1 — Recherche d'un colis par identifiant

Problème	Retrouver un colis par son ID parmi potentiellement 100 000, et ce très fréquemment (l'énoncé insiste : "l'entreprise utilise régulièrement le système pour rechercher des colis").
Solution	Maintenir, en plus de la liste des colis, un dictionnaire `id -> position` (table de hachage). La recherche consiste en une seule consultation de ce dictionnaire.
Temps	O(1) en moyenne (accès direct par hachage de la clé) ; O(n) uniquement dans le pire cas pathologique (collisions massives), négligeable en pratique avec le hachage de Python.
Espace	O(n) supplémentaire (un entier par colis dans le dictionnaire), en plus du O(n) déjà nécessaire pour stocker les colis eux-mêmes.
Alternative	Recherche linéaire dans la liste (`rechercher_lineaire_par_id`, conservée dans le code à des fins de comparaison) : O(n) dans tous les cas, O(1) espace supplémentaire.
Justification	Le coût mémoire supplémentaire (un dictionnaire de taille O(n)) est négligeable devant le gain de temps, qui devient considérable dès que n grandit — exactement le scénario visé par le TP (recherches fréquentes, grands volumes). Les mesures (section 7) confirment l'écart.

5.2 Fiche n°2 — Tri des colis pour l'affichage (Timsort)

Problème	Afficher les colis triés par poids, valeur ou priorité, sur des collections de grande taille.
Solution	`sorted(colis, key=...)`, qui utilise Timsort : l'algorithme découpe la liste en "runs" (segments déjà croissants ou décroissants), trie par insertion les runs trop courtes, puis fusionne les runs comme un tri fusion classique, avec un mode "galloping" qui accélère la fusion quand une run est beaucoup plus longue que l'autre.
Temps	O(n log n) dans le pire cas (fusion classique) ; O(n) dans le meilleur cas (données déjà triées ou presque — c'est un tri *adaptatif*).
Espace	O(n) (tableau temporaire utilisé pendant la fusion).
Alternative	Tri par tas (heapsort, O(n log n) garanti mais non stable et plus lent en pratique à cause des sauts de cache) ; tri par insertion (O(n²), inadapté dès que n dépasse quelques centaines).
Justification	Timsort est stable (deux colis de même poids/valeur/priorité gardent leur ordre d'insertion, ce qui est un comportement prévisible et souhaitable pour l'utilisateur) et offre les meilleures performances pratiques parmi les tris en O(n log n) garanti. On utilise l'implémentation native (`sorted`) car ce n'est pas le cœur algorithmique du TP, mais son fonctionnement est documenté comme demandé.

5.3 Fiche n°3 — Sélection optimale du chargement (sac à dos 0/1)

Problème	Choisir un sous-ensemble de colis dont le poids total ne dépasse pas la capacité du camion, en maximisant la valeur totale transportée. Trouver *une* combinaison valide ne suffit pas : il faut l'optimum.
Solution	Programmation dynamique exacte (`knapsack_01_dp`). Récurrence de Bellman : `dp[w] = max(dp[w], dp[w - poids_i] + valeur_i)`, calculée avec un tableau 1D "roulant" de taille `capacite + 1`, mis à jour en parcourant `w` de la capacité vers le bas pour chaque colis (garantit que chaque colis n'est utilisé qu'une seule fois). Le sous-ensemble choisi est reconstruit via une matrice compacte de booléens `keep[i][w]` (1 octet/cellule).
Temps	O(n × capacité).
Espace	O(capacité) pour le tableau `dp` ; O(n × capacité) pour la matrice de reconstruction `keep` — c'est ce terme qui devient le facteur limitant pour de grandes instances (voir section 7).
Alternative	Heuristique gloutonne par ratio valeur/poids (`knapsack_glouton_ratio`) : trie les colis par ratio décroissant et les ajoute tant qu'ils entrent. O(n log n) temps, O(n) espace — bien plus rapide, mais **non garantie optimale** en 0/1 (contrairement au cas du sac à dos fractionnaire, où ce glouton est optimal). Contre-exemple testé dans `tests/test_knapsack.py` : poids [10, 20, 30], valeurs [60, 100, 120], capacité 50 → le glouton choisit les colis 1 et 2 (valeur 160) alors que l'optimum (colis 2 et 3) vaut 220.
Justification	L'énoncé exige explicitement l'optimum ("trouver une combinaison qui respecte la capacité ne suffit pas"). La DP le garantit par construction (sous-structure optimale). Sa complexité O(n × capacité) est *pseudo-polynomiale* : elle dépend de la **valeur** de la capacité, pas de son nombre de chiffres — si la capacité devient très grande, l'algorithme devient impraticable même pour peu de colis. Le programme détecte ce cas (`SEUIL_N_CAPACITE`

	dans `knapsack.py`) et bascule automatiquement sur le glouton avec un avertissement explicite, acceptant de perdre la garantie d'optimalité au profit de la faisabilité.

5.4 Fiche n°4 — Scalabilité et choix structurels

Problème	Le système doit rester utilisable de 100 à 100 000 colis ; il faut identifier quelles opérations se dégradent et pourquoi.
Solution	Choisir, pour chaque opération, la structure dont la complexité reste acceptable à grande échelle : dictionnaire pour la recherche (O(1)), Timsort pour le tri (O(n log n)), DP avec bascule automatique pour le sac à dos (O(n × capacité) borné par un seuil).
Temps / Espace	Voir fiches 1 à 3 ; synthèse chiffrée en section 7.
Alternative	Tout faire en parcours linéaires simples (pas d'index, tri maison en O(n²)) : plus simple à écrire, mais totalement impraticable au-delà de quelques milliers de colis.
Justification	Le coût mémoire supplémentaire (dictionnaire d'index, tableau de reconstruction du sac à dos) est le prix à payer pour rester rapide à grande échelle ; le programme le paie seulement là où c'est nécessaire (le glouton de secours évite de payer le coût mémoire O(n × capacité) quand ce n'est plus raisonnable).

6. Variante : favoriser les colis prioritaires

Aucune formule n'étant imposée, la règle suivante a été choisie :

```
score_normalise(colis)  = (priorite - 1) / 4        # 0.0 (prio 1) .. 1.0 (prio 5)
valeur_ajustee(colis)   = valeur * (1 + alpha * score_normalise)
```

avec `alpha = 0.3` par défaut (réglable dans le menu). On relance **exactement le même algorithme** `knapsack_01_dp`, en lui donnant la valeur ajustée comme "valeur" à maximiser, mais on affiche toujours à l'utilisateur la **vraie valeur réelle** (`valeur_totale_reelle`) des colis sélectionnés — jamais la valeur ajustée, qui n'est qu'un outil interne pour orienter le choix.

Exemple chiffré (`tests/test_knapsack.py::test_priorite_peut_changer_la_selection`) : deux colis de même poids (10 kg), capacité 10 kg (un seul peut être chargé) :

Colis	Poids	Valeur	Priorité	Valeur ajustée (α=0.3)
1	10	100	1 (peu important)	100 × 1.0 = 100
2	10	90	5 (très important)	90 × 1.3 = 117

- Sans priorité : la DP choisit le colis 1 (valeur 100 > 90).
- Avec priorité : la DP choisit le colis 2 (valeur ajustée 117 > 100), bien qu'il transporte réellement moins de valeur (90 < 100) — c'est l'effet recherché : favoriser l'important même au prix d'un peu de valeur brute.

`alpha` contrôle l'intensité de cet effet : `alpha = 0` revient à ignorer totalement la priorité (comportement identique à la section 5.3) ; plus `alpha` est grand, plus la priorité peut l'emporter sur l'écart de valeur brute entre deux colis.

7. Résultats de benchmark (mesures réelles)

Script : `benchmarks/run_benchmarks.py`, exécuté sur la machine de développement. Résultats complets dans `benchmarks/results/benchmark_results.csv`.

n	recherche (dict)	recherche (linéaire)	tri par poids	camion, capacité 200	camion, capacité 1000
100	0,083 µs	0,64 µs	0,012 ms	0,684 ms (DP)	4,86 ms (DP)
1 000	0,076 µs	5,49 µs	0,102 ms	5,54 ms (DP)	35,8 ms (DP)
10 000	0,108 µs	49,7 µs	0,979 ms	48,8 ms (DP)	1,45 ms (**glouton**, n×cap = 10 000 000 > seuil)
100 000	0,50 µs	492 µs	9,72 ms	18,0 ms (**glouton**, n×cap = 20 000 000 > seuil)	20,6 ms (**glouton**, n×cap = 100 000 000 > seuil)

(temps moyennés sur 200 recherches pour les deux premières colonnes ; seuil de bascule DP → glouton fixé à `n × capacité ≤ 3 000 000`.)

Analyse — deux opérations dont le coût croît avec n

1. Recherche linéaire par ID — O(n). Le temps mesuré croît proportionnellement à n (0,64 µs → 5,49 µs → 49,7 µs → 492 µs, soit un facteur ≈10 à chaque fois que n est multiplié par 10, conforme à O(n)). À 100 000 colis, une recherche prend déjà ~500 µs ; pour un système "utilisé régulièrement" comme le décrit l'énoncé, cela devient perceptible dès que l'utilisateur recherche plusieurs colis d'affilée. **Ce choix ne serait plus adapté seul** : c'est précisément pour cela que le programme utilise par défaut la version indexée par dictionnaire (colonne "recherche (dict)"), qui reste quasi constante (0,08–0,5 µs, aucune croissance visible significative sur la plage testée) — **amélioration déjà en place** dans le code livré ; la version linéaire n'est conservée que pour la démonstration et le benchmark.

2. Préparation du camion (DP) — O(n × capacité). Le temps de la DP exacte croît avec le produit n × capacité : à capacité fixe, il est multiplié par ~10 quand n l'est (0,684 ms → 5,54 ms → 48,8 ms pour capacité 200). Mais à partir de n = 10 000 avec capacité 1 000 (produit 10 000 000), puis dès n = 100 000 même à capacité 200 (produit 20 000 000), le produit dépasse le seuil retenu et l'algorithme exact devient trop coûteux en mémoire pour la matrice de reconstruction (O(n × capacité) octets). **Le choix de la DP seule ne reste pas adapté** au-delà de ce seuil : **l'amélioration mise en place** est la bascule automatique vers l'heuristique gloutonne (O(n log n)), qui reste rapide (quelques millisecondes même à 100 000 colis) au prix de perdre la garantie d'optimalité — compromis explicitement signalé à l'utilisateur par un avertissement.

Autres pistes d'amélioration (non implémentées)

- Reconstruire le sous-ensemble choisi sans stocker la matrice complète O(n × capacité) (technique de reconstruction "en place" par un second passage arrière sur un tableau 1D, ou approche diviser-pour-régner à la Hirschberg) : réduirait l'espace à O(capacité), repoussant le seuil de bascule vers le glouton.
- Pour des capacités très grandes, une approximation FPTAS (arrondi des valeurs à une précision contrôlée) permettrait de garder une garantie d'approximation tout en bornant le temps de calcul, sans tomber directement sur le glouton pur.

8. Tests

`tests/` contient des tests `pytest` couvrant : - la validation du modèle (`test_models.py`) ; - la cohérence recherche dict / recherche linéaire et la gestion des doublons d'ID (`test_repository.py`) ; - le tri par les trois critères et sa stabilité (`test_tri.py`) ; - le cas connu de l'énoncé (colis A/B/C/D, capacité 14 → optimum {A, B}, valeur 45, vérifié à la main), la non-optimalité du glouton sur un contre-exemple classique, l'effet de la priorité sur la sélection, et la garantie que la valeur affichée est toujours la valeur réelle (jamais la valeur ajustée) (`test_knapsack.py`).

Exécution : `.venv/bin/python -m pytest tests/ -v` — 20 tests, tous passants.

9. Conclusion

Le système répond aux exigences du TP : gestion complète de la collection de colis avec une recherche en O(1) moyen, des tris en O(n log n), une sélection optimale du chargement par programmation dynamique garantie optimale pour des instances raisonnables, une variante prenant en compte la priorité sans dupliquer d'algorithme, et une bascule automatique vers une heuristique rapide quand l'instance dépasse ce que la DP peut traiter en mémoire/temps raisonnable. Les mesures réelles aux quatre échelles demandées confirment les analyses de complexité théoriques et motivent les choix de conception retenus.